PROJECT SYNOPSIS

Smart Multi-Level Parking Management System

ParkEase — a time-based, conflict-free parking reservation platform

This document distinguishes what is built in the current working prototype from what is proposed as future scope. Each feature below is marked IMPLEMENTED or PLANNED so the reader knows exactly what can be demonstrated today.

1. Introduction

The rapid increase in the number of vehicles has created significant challenges in managing parking spaces efficiently. Traditional parking systems often depend on manual processes, provide limited information about slot availability, and make it difficult for users to reserve parking in advance. Users may also face problems when they need to extend their parking duration, or when a previously selected slot has already been reserved by another vehicle for a later time.

The proposed Smart Multi-Level Parking Management System, ParkEase, is a web-based parking management platform designed to provide an organized, convenient, and time-based parking experience. The system divides the parking facility into dedicated levels according to vehicle type: the basement for petrol cars, the first floor for two-wheelers, and the second floor for electric vehicles with charging facilities.

A working frontend prototype of this system has been built and is demonstrable end-to-end: a user can select a vehicle type, choose a date and time window, view live slot availability for that exact window, book a slot, extend an active booking, and view booking history — all running in the browser. The prototype currently stores data locally in the browser rather than in a shared backend database; Section 7 explains this distinction and the plan to move to a full client-server architecture.

The system is designed to be adaptable to different environments such as shopping malls, offices, residential complexes, educational institutions, airports, and hospitals. Section 4 discusses the hospital use case in detail as an illustration of how the platform's design generalises beyond a single facility type; it is a proposed application of the architecture rather than a feature implemented in the current prototype.

2. Problem Statement

Conventional parking systems have several limitations, including difficulty in finding available spaces, lack of convenient advance reservation, inefficient management of parking slots, and conflicts between existing and future bookings. Most existing systems mark a slot as simply "available" or "occupied" for the entire day, with no awareness of specific time windows — so a slot that becomes free at 1 PM cannot be booked by a user arriving at 10 AM.

A particular challenge occurs when a user wants to extend their parking duration while another user has already reserved the same slot for a later time. Without a time-based reservation system that checks for overlap before confirming a booking, such conflicts are difficult to detect and can lead to disputes at the gate.

These problems become more significant in high-traffic locations such as hospitals, where patients, attendants, doctors, staff, and visitors may need parking at different times, and emergency situations may require quick identification of parking close to hospital entrances. There is also a growing need for dedicated EV parking and charging facilities. A centralized system is therefore required to manage different vehicle categories, parking slots, reservations, and extensions in a way that prevents scheduling conflicts.

3. Objectives

- To provide a centralized digital parking management platform.
- To divide parking spaces according to vehicle type.
- To allow users to check parking availability for a specific date and time window, not just the whole day.
- To provide time-based pre-booking of parking slots.
- To prevent overlapping reservations for the same slot through automatic conflict detection.
- To allow users to extend parking time when the slot remains available for the extended period.
- To direct users to alternative slots when an extension is not possible.
- To distinguish normal users from members, with membership adding benefits rather than geting core booking functionality.
- To maintain a record of parking history for each user.
- To provide dedicated EV parking, and, in a later phase, EV charging management.
- To design the system so its architecture can be adapted to hospitals and other high-traffic facilities.
- To extend the current frontend prototype into a full client-server system with authentication, a shared database, and an admin dashboard.

4. Proposed System

The proposed system consists of three dedicated parking levels, implemented and demonstrable in the current prototype:

Basement — Petrol Cars [IMPLEMENTED]
Dedicated parking slots for petrol-powered cars.

Floor 1 — Two-Wheelers [IMPLEMENTED]
Dedicated parking slots for motorcycles and scooters.

Floor 2 — Electric Vehicles [IMPLEMENTED]
Dedicated EV parking spaces. Slot allocation for this level is implemented; charging-specific hardware integration and charging-based pricing are planned (see Section 10).

4.1 Time-Based Booking and Conflict Detection

Users select their vehicle type, date, entry time, and expected exit time. The system displays available slots for exactly that period rather than for the whole day. This time-window model, and the conflict check that enforces it, are implemented in the current prototype.

The system uses time-based booking rather than marking a slot as permanently occupied. For example, if Slot B12 is booked from 10:00 AM to 1:00 PM, another user can book the same slot from 1:00 PM to 4:00 PM, provided the two time windows do not overlap. A new booking is accepted only if:

New Start < Existing End AND New End > Existing Start → Conflict

If both conditions are true for any existing confirmed booking on that slot, the new booking is rejected with a clear message naming the conflicting time window.

When a user requests an extension of an active booking, the system checks only the gap between the current exit time and the newly requested exit time against other reservations on that slot. If the requested extension conflicts with another booking, the extension is rejected and the user is directed back to the slot-selection screen to choose another slot. Automatic suggestion of a specific alternative slot, rather than directing the user back to the slot grid, is listed as future scope in Section 10.

4.2 Hospital Application (Proposed Extension)

The same underlying architecture — vehicle-type levels, time-window booking, and conflict detection — could be adapted for hospital parking management. This is a proposed application to illustrate the platform's generality; the categories below are not implemented as distinct roles in the current prototype, which supports vehicle-type-based levels only.

- Emergency vehicles and ambulances.
- Patients and their attendants.
- Doctors and medical staff.
- Hospital employees.
- General visitors.

For example, a hospital could reserve specific slots near the emergency entrance for ambulances while managing regular visitor and staff parking through the same platform, and users could pre-book parking before arriving to reduce time spent searching for a space during an appointment. Implementing this would require adding a role/category model on top of the existing vehicle-type model, which is scoped as future work.

5. Membership System

Pre-booking is available to all users without a membership. The membership system provides additional benefits rather than restricting basic parking functionality — this separation is implemented in the current prototype as two user tiers: Normal and Member.

- [IMPLEMENTED] Discounted parking charges for members.
- [IMPLEMENTED] Priority/member parking slots, visually distinguished in the slot grid.
- [IMPLEMENTED] An extended pre-booking window for members (booking further in advance than normal users).
- [IMPLEMENTED] Reduced cancellation charges for members.
- [IMPLEMENTED] Loyalty/reward points accrued on paid bookings.
- [IMPLEMENTED] An extended grace period after the scheduled exit time before overstay charges apply.

- [PLANNED] EV-charging-specific discounts, separate from the general membership discount.
- [PLANNED] Monthly parking passes as a distinct purchasable product.

Only two membership tiers are used in the current prototype — Normal and Member — kept deliberately simple rather than offering several overlapping paid tiers.

6. Key Features

- Multi-level parking management by vehicle type — IMPLEMENTED
- Time-based slot availability for a specific date and time window — IMPLEMENTED
- Future date and time-based pre-booking — IMPLEMENTED
- Slot selection and reservation — IMPLEMENTED
- Automatic booking conflict detection — IMPLEMENTED
- Parking extension with conflict re-checking — IMPLEMENTED
- Redirect to slot selection when an extension is blocked — IMPLEMENTED
- Grace period and overstay charge calculation — IMPLEMENTED
- No-show detection with a fixed check-in window — IMPLEMENTED
- Simulated online payment and digital parking receipt (QR-based) — IMPLEMENTED
- Parking history per user — IMPLEMENTED
- Two-tier membership with discounts, priority slots, and loyalty points — IMPLEMENTED
- User registration and authentication — PLANNED
- Shared backend database (multi-device, multi-user data) — PLANNED
- Admin dashboard for facility operators — PLANNED
- Automatic suggestion of a specific alternative slot on conflict — PLANNED
- Dedicated EV charging session management and pricing — PLANNED
- Role-based categories for hospital, office, or campus deployments — PLANNED
- Monthly parking passes — PLANNED

7. Technology Stack

The current prototype is a frontend-only implementation. All booking logic, conflict detection, extension handling, grace-period and no-show calculation, and membership rules run client-side in the browser, and data is persisted using the browser's local storage rather than a server-side database. This was a deliberate choice for this stage of the project, to validate the booking and conflict-detection logic quickly without backend infrastructure. It means data does not currently sync across devices or persist if browser storage is cleared.

Layer	Current Prototype	Planned (Full System)
Frontend	HTML, CSS, vanilla JavaScript	React.js, Tailwind CSS (or continue with the current stack, if preferred)
Backend	None — logic runs entirely in the	Node.js, Express.js
Data storage	Browser local storage (per device, not shared)	MongoDB (shared, multi-user, multi-device)
Authentication	None	Token-based user authentication (e.g. JWT)

Recommendation: either implement the Node.js / Express.js / MongoDB backend before final submission so the architecture diagram and this table match a working system, or keep the stack

described above as-is and present the project explicitly as a validated frontend prototype with a defined backend plan. Claiming the full stack without a backend to show risks a credibility question during evaluation.

8. System Workflow

8.1 Core Booking Workflow (Implemented)

Select Vehicle Type → Select Date, Entry Time & Exit Time → View Available Slots for That Window → Select Slot → Confirm Booking → Pay → Digital Receipt (QR) → Check In → Park → Extend if Required → Exit

8.2 Extension Workflow (Implemented)

Active or Upcoming Booking → Select New Exit Time → Check for Conflict on the Same Slot → If Available: Confirm Extension | If Unavailable: Show Conflicting Booking and Direct to Slot Selection

8.3 Registration and Hospital Workflow (Planned)

User Registration → Vehicle Selection → Select Date & Time → Check Slot Availability → Select Slot → Confirm Booking → Parking → Extend if Required → Exit → Payment → Digital Receipt → Parking History

For hospital use: Patient/Visitor → Select Hospital Parking Category → Select Vehicle → Choose Date & Time → Reserve Slot → Navigate to Parking → Park → Exit. This workflow depends on the role-based category model described in Section 10 and is not part of the current prototype.

9. Applications

The vehicle-type-based level structure and time-window booking engine are general enough to be adapted for multiple environments. The following are proposed applications of the platform, not separate implementations:

- Hospitals: patient, visitor, staff, and emergency parking (requires the role-based category model in Section 10).
- Shopping malls: customer parking and advance reservation.
- Educational institutions: student and faculty parking.
- Corporate offices: employee and visitor parking.
- Residential complexes: resident and guest parking.
- Airports: long-term and short-term parking management.
- Public parking facilities: centralized digital parking management.

10. Future Scope

Beyond the specific gaps already noted in Sections 5–9, the system can be further enhanced with:

- A full backend (Node.js, Express.js, MongoDB) to replace local-storage persistence with shared, multi-device data.
- User registration and authentication.
- An admin dashboard for facility operators to view and manage all bookings, slots, and users.
- Role-based parking categories (e.g. for hospital deployments: emergency, patient, staff, visitor).
- Automatic suggestion of a specific alternative slot when a conflict is detected, rather than redirecting to the slot grid.
- Dedicated EV charging session management and charging-specific pricing.
- Monthly parking passes as a purchasable product.
- IoT-based parking sensors for real hardware-verified occupancy.
- Automatic Number Plate Recognition (ANPR) and automated entry/exit gates.
- A native mobile application.
- AI-based parking demand prediction and dynamic pricing.
- Indoor navigation to assigned parking slots.
- Integration with hospital appointment systems for pre-arrival parking booking.
- Cloud-based management across multiple parking facilities.

11. Expected Outcome

The working prototype already demonstrates the core contribution of this project: time-window-based slot booking with automatic conflict detection, which most existing parking systems lack. It shows, end-to-end, that a slot can be safely shared between multiple users across a day, that extensions can be checked safely against future reservations, and that membership can add value without gating basic access.

Once extended with the planned backend, authentication, and admin dashboard, the system is expected to provide a more organized and convenient method of managing multi-level parking facilities at production scale, reducing time spent searching for parking, minimizing booking conflicts, and supporting structured management of different vehicle categories. Adapting it to hospitals and other high-traffic facilities would additionally require the role-based category model described in Section 10, but the underlying booking and conflict-detection engine already validated in this prototype would carry over directly.

12. Conclusion

The Smart Multi-Level Parking Management System (ParkEase) provides a centralized approach to managing parking spaces in a multi-level facility, combining vehicle-specific parking levels, time-based pre-booking, and automatic conflict detection. These core mechanics are implemented and demonstrable in the current frontend prototype, distinguishing this project from parking systems that only track whole-day occupancy.

The remaining components — a shared backend, authentication, an admin dashboard, EV charging management, monthly passes, and role-based hospital categories — are well-defined as future work rather than open-ended ambitions, and build directly on the booking and conflict-detection engine already validated. This gives the project a clear, honest line between what has been built and what is proposed next.

Submitted By:

Devanshu Shankar - 10324210010
Shorya Bansal -10324210012
Kartik Gupta -10324210023

Under the Guidance of :
MR. Jitendra Varshney

Department of Computer Science & Engineering
Academic Year: 2026–27

SRM UNIVERSITY SONPEAT, DELHI NCR.